

# ■ ALGOLENS

AI Driven Code Performance Analyzer

## ANALYSIS REPORT

| Overall Score | Quality Score | Performance Score |
|---------------|---------------|-------------------|
| 86/100        | 90/100        | 80/100            |

### 1. Code Metrics

| Metric        | Value |
|---------------|-------|
| Lines of Code | 6     |
| Functions     | 1     |
| Loops         | 2     |
| Conditions    | 0     |
| Variables     | 1     |

### 2. Performance Analysis

| Metric             | Result   |
|--------------------|----------|
| Time Complexity    | $O(n^2)$ |
| Space Complexity   | $O(n)$   |
| Maximum Loop Depth | 2        |
| Performance Score  | 80/100   |

### 3. Code Quality

Quality Score: 90/100

- Nested loops detected.

### 4. Performance Issues

- Nested loops detected. Performance may decrease for large inputs.
- Additional list memory usage detected.

### 5. AI Code Summary

#### ### 1. What the code does

The code generates and prints all possible ordered pairs (Cartesian product) of the numbers in a given list, including pairing each number with itself.

---

#### ### 2. Main purpose of the code

Its main purpose is to demonstrate the use of nested loops to compare or combine every element of a sequence with every element in that same sequence.

---

#### ### 3. How the code works

1. **Function Definition:** `print_pairs(numbers)` takes a list as an argument.
2. **Outer Loop:** Iterates through each element `i` in the list one by one.
3. **Inner Loop:** For every single iteration of the outer loop, the inner loop iterates through the entire list again to pick an element `j`.
4. **Print:** Prints the current pair `(i, j)`.
5. **Execution:** The list `[1, 2, 3]` is passed to the function, printing 9 pairs in total (e.g., `1 1`, `1 2`, `1 3`, `2 1`, etc.).

---

#### ### 4. Important observations

- \* **Time Complexity:**  $O(N^2)$ , where  $N$  is the number of elements in the list. Performance degrades rapidly as the list size grows because of the nested loops.
- \* **Space Complexity:**  $O(1)$  auxiliary space, as it prints pairs directly without storing them in memory.
- \* **Redundancy:** It prints self-paired items (like `1 1`) as well as symmetric pairs in both orders (like `1 2` and `2 1`).

## 6. AI Improvement Suggestions

AI suggestions could not be generated.

## 7. AI Improved Code

```
from itertools import product
from typing import Any, Iterable

def print_pairs(numbers: Iterable[Any]) -> None:
    for i, j in product(numbers, repeat=2):
        print(i, j)

if __name__ == "__main__":
    numbers = [1, 2, 3]
    print_pairs(numbers)
```